

Classes That Were Useful

1. Advanced Algorithmics and Data Structures

- **How it was useful:** This course was the foundation of our entire dispatch engine. It allowed us to analyze the computational complexity of the Vehicle Routing Problem with Time Windows (VRPTW). Understanding search space limitations guided us to design a custom, structured greedy heuristic rather than an unmanageable brute-force search. We used data structures like hash maps for the Paris Travel-Time Matrix and custom priority lists to sort deliveries based on critical medical urgency.

2. AI for Software Development

- **How it was useful:** This module helped us bridge the gap between our current prototype and future system scaling. We applied this knowledge directly when drafting the Product Requirements Document (PRD) to outline how Generative AI (LLMs) can be integrated. Specifically, it guided how we structured the prompt-engineering logic for parsing unstructured, natural-language client order emails and converting them into structured, validation-ready JSON objects.

3. Intermediate Python

- **How it was useful:** The entire functional prototype was built in Python, making this class directly applicable. We used advanced Python concepts such as @dataclass decorators for modeling real-world logistics objects, complex dictionary structures for traffic matrices, list comprehensions for dynamic filtering, and deep copying (copy.deepcopy) to safely simulate disruptions without corrupting our baseline data.

4. Introduction to Object-Oriented Programming (OOP)

- **How it was useful:** OOP principles allowed us to write clean, modular, and maintainable code. We modeled real-world entities—such as vehicles and deliveries—as distinct classes with protected properties (like current load, capacities, and driver shift limits). Encapsulating the validation logic within these objects made it easy to run complex constraint checks (check_all_constraints) without cluttering the main program loop.

5. Shell Programming on Linux

- **How it was useful:** This course was highly practical for setting up and managing our local development space. It helped us write setup and execution scripts, and resolve dependency conflicts when deploying the Streamlit application. Understanding shell allowed us to cleanly execute and debug the local server.

6. Web Programming

- **How it was useful:** Although Streamlit abstracts away much of the traditional web-app boilerplate, understanding client-server architecture, stateless protocols, and rendering loops was essential. It allowed us to master Streamlit's execution cycle and implement state persistence (`st.session_state`) to handle user inputs (like sliders and checkboxes) without resetting the core routing calculations.